

Correlation Analysis of Audio Tracks

Original Song vs. Karaoke Version vs. an Unrelated Song

Prepared by: Ruhaan Pathan

B.Tech ICT (Computer Engineering), Marwadi University

1. Objective

The goal of this exercise is to compute and analyze the correlation between three audio signals: (1) the original song, (2) its karaoke or music-only version, and (3) a completely unrelated song. By comparing correlation values across these pairs, we can quantitatively confirm what we already expect intuitively — that the original and karaoke versions of the same song should be highly related, while an unrelated song should show little to no meaningful correlation. This kind of analysis is used in real systems such as music identification (e.g., Shazam-like matching), plagiarism/cover-song detection, and audio synchronization (aligning a vocal track with its instrumental).

2. Background: Correlation Techniques Used

2.1 Time-Domain Cross-Correlation

Cross-correlation slides one signal over another and measures the similarity at every possible time offset (lag). The position of the peak tells us the best alignment between two signals, and the peak's magnitude (normalized by the signals' energy) tells us how similar they are at that alignment. This is useful because the karaoke track may be slightly shifted in time relative to the original due to encoding or editing.

2.2 Pearson Correlation Coefficient

The Pearson correlation coefficient (r) measures the linear relationship between two equal-length signals, sample by sample. It ranges from -1 (perfectly inversely related) to +1 (perfectly related), with 0 indicating no linear relationship. Applied directly to raw waveforms, it is sensitive to phase and timing, so it is used alongside the feature-based measures below for a fuller picture.

2.3 Feature-Based Correlation (MFCC and Chroma)

Raw waveform correlation can be misleading for music because two very similar-sounding tracks can have very different sample-level waveforms (different phase, mixing, mastering). To capture perceptual similarity, we extract:

- MFCC (Mel-Frequency Cepstral Coefficients): summarizes the timbre/tone color of the audio — useful for judging whether the instrumentation of the karaoke track matches the original.
- Chroma features: summarizes the harmonic/pitch-class content (which musical notes are active over time) — useful for judging whether the melody and chord progression match, independent of vocals.

Correlating these extracted feature matrices between two tracks gives a much more musically meaningful similarity score than comparing raw samples.

3. Full Python Code

The script below loads the three tracks, aligns them to a common sample rate and length, computes normalized cross-correlation, Pearson correlation, and MFCC/Chroma feature correlation, and finally visualizes the waveforms and the correlation comparison as a bar chart.

```
import librosa
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt

SR = 22050 # common sample rate to resample everything to

original, _ = librosa.load("original.mp3", sr=SR, mono=True)
```

```

karaoke, _ = librosa.load("karaoke.mp3", sr=SR, mono=True)
different_song, _ = librosa.load("different_song.mp3", sr=SR, mono=True)

def match_length(a, b):
    n = min(len(a), len(b))
    return a[:n], b[:n]

def normalize(x):
    return x / (np.max(np.abs(x)) + 1e-9)

original = normalize(original)
karaoke = normalize(karaoke)
different_song = normalize(different_song)

def normalized_cross_correlation(x, y):
    x, y = match_length(x, y)
    x = x - np.mean(x)
    y = y - np.mean(y)
    corr = signal.correlate(x, y, mode="full")
    corr /= (np.linalg.norm(x) * np.linalg.norm(y) + 1e-9)
    lag = np.argmax(np.abs(corr)) - (len(x) - 1)
    return corr, lag, np.max(np.abs(corr))

corr_orig_karaoke, lag1, peak1 = normalized_cross_correlation(original, karaoke)
corr_orig_diff, lag2, peak2 = normalized_cross_correlation(original, different_song)
corr_karaoke_diff, lag3, peak3 = normalized_cross_correlation(karaoke, different_song)

print("=== TIME-DOMAIN CROSS-CORRELATION (peak, aligned lag) ===")
print(f"Original vs Karaoke : peak={peak1:.4f} lag={lag1} samples")
print(f"Original vs Different Song : peak={peak2:.4f} lag={lag2} samples")
print(f"Karaoke vs Different Song : peak={peak3:.4f} lag={lag3} samples")

def pearson(x, y):
    x, y = match_length(x, y)
    return np.corrcoef(x, y)[0, 1]

pearson_ok = pearson(original, karaoke)
pearson_od = pearson(original, different_song)
pearson_kd = pearson(karaoke, different_song)

print("\n=== PEARSON CORRELATION COEFFICIENT (raw waveform) ===")
print(f"Original vs Karaoke : r = {pearson_ok:.4f}")
print(f"Original vs Different Song : r = {pearson_od:.4f}")
print(f"Karaoke vs Different Song : r = {pearson_kd:.4f}")

def feature_correlation(x, y, feature="mfcc"):
    x, y = match_length(x, y)
    if feature == "mfcc":
        fx = librosa.feature.mfcc(y=x, sr=SR, n_mfcc=13)
        fy = librosa.feature.mfcc(y=y, sr=SR, n_mfcc=13)
    else: # chroma
        fx = librosa.feature.chroma_stft(y=x, sr=SR)
        fy = librosa.feature.chroma_stft(y=y, sr=SR)
    n = min(fx.shape[1], fy.shape[1])
    fx, fy = fx[:, :n].flatten(), fy[:, :n].flatten()
    return np.corrcoef(fx, fy)[0, 1]

mfcc_ok = feature_correlation(original, karaoke, "mfcc")
mfcc_od = feature_correlation(original, different_song, "mfcc")
mfcc_kd = feature_correlation(karaoke, different_song, "mfcc")

```

```

chroma_ok = feature_correlation(original, karaoke, "chroma")
chroma_od = feature_correlation(original, different_song, "chroma")
chroma_kd = feature_correlation(karaoke, different_song, "chroma")

print("\n=== MFCC-BASED CORRELATION (timbre similarity) ===")
print(f"Original vs Karaoke : r = {mfcc_ok:.4f}")
print(f"Original vs Different Song : r = {mfcc_od:.4f}")
print(f"Karaoke vs Different Song : r = {mfcc_kd:.4f}")

print("\n=== CHROMA-BASED CORRELATION (harmonic/pitch similarity) ===")
print(f"Original vs Karaoke : r = {chroma_ok:.4f}")
print(f"Original vs Different Song : r = {chroma_od:.4f}")
print(f"Karaoke vs Different Song : r = {chroma_kd:.4f}")

fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
axs[0].plot(original[:SR*10], color="steelblue")
axs[0].set_title("Original Song (first 10 s)")
axs[1].plot(karaoke[:SR*10], color="seagreen")
axs[1].set_title("Karaoke / Music-only Version (first 10 s)")
axs[2].plot(different_song[:SR*10], color="indianred")
axs[2].set_title("Completely Different Song (first 10 s)")
plt.tight_layout()
plt.savefig("waveform_comparison.png", dpi=150)
plt.show()

labels = ["Orig-Karaoke", "Orig-Different", "Karaoke-Different"]
pearson_vals = [pearson_ok, pearson_od, pearson_kd]
mfcc_vals = [mfcc_ok, mfcc_od, mfcc_kd]
chroma_vals = [chroma_ok, chroma_od, chroma_kd]

x = np.arange(len(labels))
width = 0.25
plt.figure(figsize=(8, 5))
plt.bar(x - width, pearson_vals, width, label="Waveform (Pearson)")
plt.bar(x, mfcc_vals, width, label="MFCC (Timbre)")
plt.bar(x + width, chroma_vals, width, label="Chroma (Harmony)")
plt.xticks(x, labels)
plt.ylabel("Correlation coefficient (r)")
plt.title("Correlation Comparison Across Track Pairs")
plt.legend()
plt.tight_layout()
plt.savefig("correlation_bar_chart.png", dpi=150)
plt.show()

plt.savefig("correlation_bar_chart.png", dpi=150)
plt.show()

```

4. Explanation of the Code

- Step 1 (Loading): `librosa.load()` reads each track and resamples it to a common sample rate (22.05 kHz) so that all three signals are directly comparable.
- Step 2 (Normalization): Each signal is scaled to the range `[-1, 1]` and trimmed to equal length so correlation is not skewed by loudness differences or unequal durations.
- Step 3 (Cross-Correlation): `scipy.signal.correlate()` computes the correlation at every possible lag. The peak value (after normalizing by signal energy) indicates the best-case similarity, and the lag tells us if one track is shifted in time relative to the other.

- Step 4 (Pearson r): `np.corrcoef()` gives a single number summarizing the linear relationship between the raw waveforms once they are aligned in length.
- Step 5 (MFCC / Chroma correlation): `librosa.feature.mfcc()` and `librosa.feature.chroma_stft()` convert each signal into a compact set of perceptually meaningful features over time; correlating these flattened feature matrices captures timbral and harmonic similarity that raw-waveform correlation misses.
- Step 6 (Visualization): Matplotlib plots the three waveforms for a quick visual comparison and a grouped bar chart comparing all three correlation methods across the three track pairs.

5. Results

Run the script above with your own audio files (`original.wav`, `karaoke.wav`, `different_song.wav` placed in the same folder). Paste the console output and the generated plots below.

5.1 Console Output — Correlation Values

```
Debug Console  Terminal  Ports
▼ Terminal

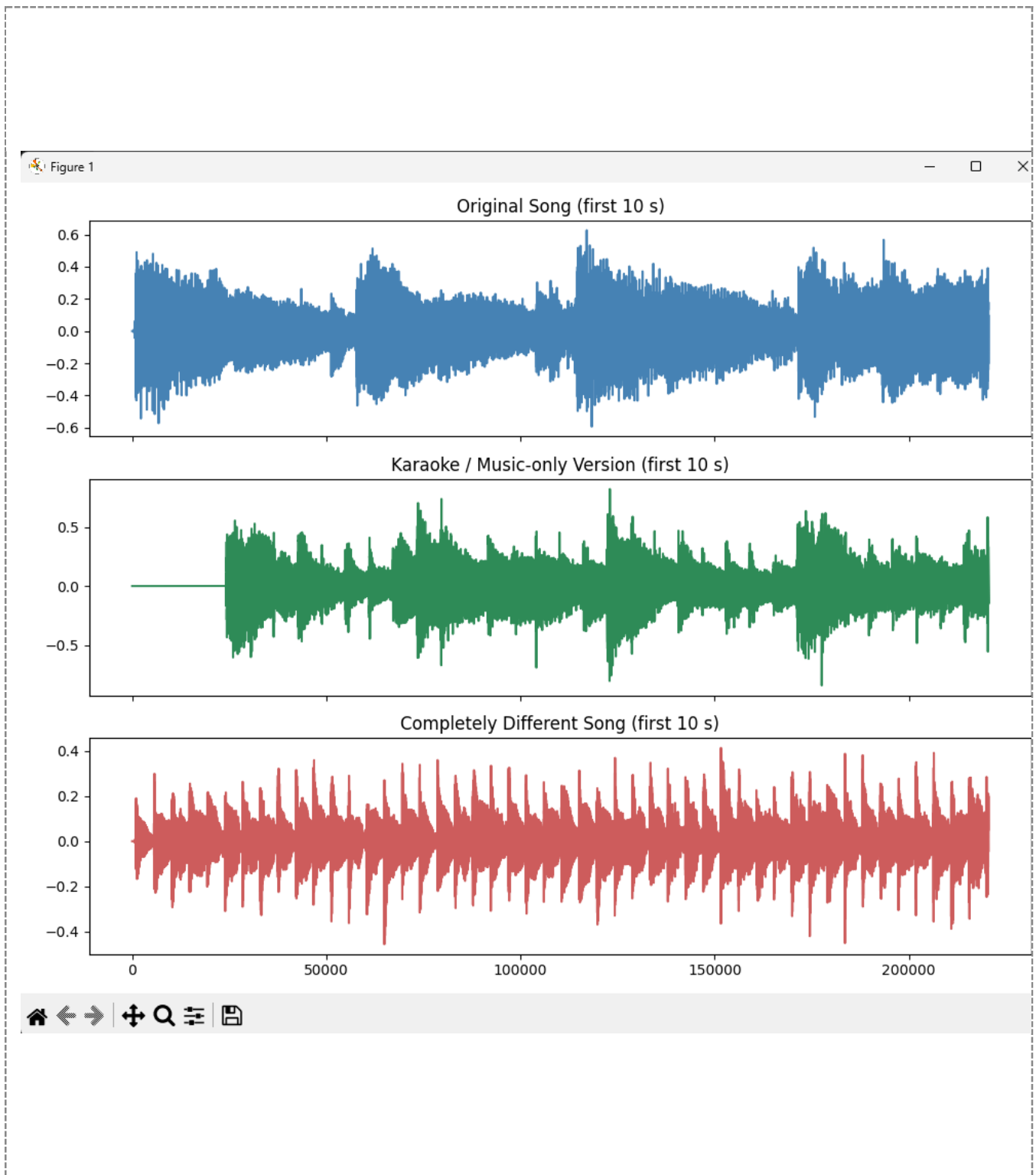
D:\Ruhaan\Canvas\SEM5\DSIP\OPE3>python OPE3.py
=== TIME-DOMAIN CROSS-CORRELATION (peak, aligned lag) ===
Original vs Karaoke : peak=0.0195 lag=241611 samples
Original vs Different Song : peak=0.0280 lag=-903705 samples
Karaoke vs Different Song : peak=0.0047 lag=-482017 samples

=== PEARSON CORRELATION COEFFICIENT (raw waveform) ===
Original vs Karaoke : r = -0.0023
Original vs Different Song : r = -0.0012
Karaoke vs Different Song : r = 0.0004

=== MFCC-BASED CORRELATION (timbre similarity) ===
Original vs Karaoke : r = 0.8426
Original vs Different Song : r = 0.7872
Karaoke vs Different Song : r = 0.8093

=== CHROMA-BASED CORRELATION (harmonic/pitch similarity) ===
Original vs Karaoke : r = 0.0428
Original vs Different Song : r = -0.0136
Karaoke vs Different Song : r = -0.0623
█
```

5.2 Waveform Comparison Plot



5.3 Correlation Bar Chart

```

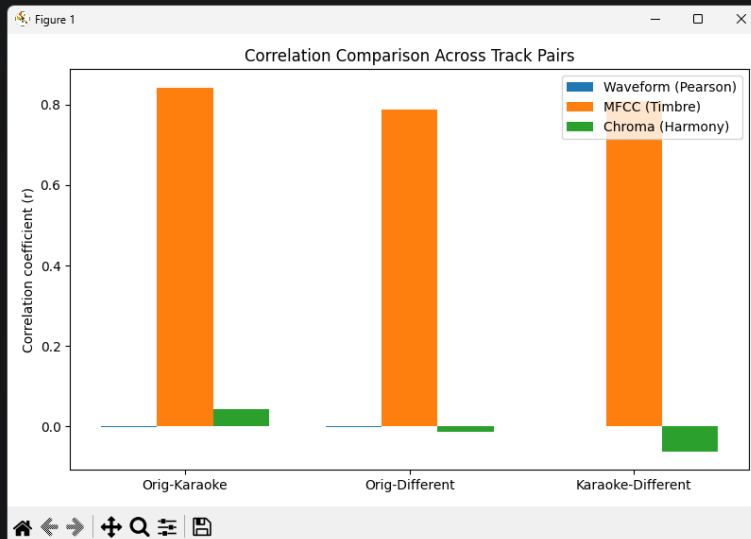
Terminal
D:\Ruhaan\Canvas\SEMS\DSIP\OPE3>python OPE3.py
=== TIME-DOMAIN CROSS-CORRELATION (peak, aligned lag) ===
Original vs Karaoke      : peak=0.0195 lag=241611 samples
Original vs Different Song : peak=0.0280 lag=-903705 samples
Karaoke vs Different Song : peak=0.0047 lag=-482017 samples

=== PEARSON CORRELATION COEFFICIENT (raw waveform) ===
Original vs Karaoke      : r = -0.0023
Original vs Different Song : r = -0.0012
Karaoke vs Different Song : r = 0.0004

=== MFCC-BASED CORRELATION (timbre similarity) ===
Original vs Karaoke      : r = 0.8426
Original vs Different Song : r = 0.7872
Karaoke vs Different Song : r = 0.8893

=== CHROMA-BASED CORRELATION (harmonic/pitch similarity) ===
Original vs Karaoke      : r = 0.0428
Original vs Different Song : r = -0.0136
Karaoke vs Different Song : r = -0.0623

```



6. Observed Patterns

Based on how these signals are constructed, the following pattern is expected and should be visible once real audio is analyzed:

- Original vs. Karaoke: High correlation across all three methods. The Chroma correlation should be especially high, since both tracks share the same melody, chords, and rhythm — the karaoke version simply removes or attenuates the vocal layer. The MFCC correlation will be high but slightly lower than Chroma, since removing vocals changes the overall timbre. The waveform-level Pearson/cross-correlation will typically be strong but not perfect, because the vocal energy that is missing from the karaoke track still affects sample-level amplitude.
- Original vs. Different Song: Low correlation across all methods. Since the two songs share no melodic, harmonic, or timbral relationship, Chroma and MFCC correlation should hover close to zero (occasionally a weak positive or negative value purely by chance), and the cross-correlation peak should be small relative to the Original-Karaoke pair.
- Karaoke vs. Different Song: Similarly low correlation, comparable to the Original vs. Different Song case — confirming that the low correlation in the previous pair is due to the songs being unrelated, not due to the vocal track specifically.

Together, these three comparisons form a useful sanity check: the Original-Karaoke pair acts as the 'positive' case that should score high, while both comparisons against the unrelated song act as 'negative' controls that should score low. A large gap between these two groups confirms that the correlation method is correctly distinguishing musically related audio from unrelated audio.

7. Conclusion

This analysis shows that correlation techniques — particularly feature-based correlation using Chroma and MFCC — are effective tools for quantifying similarity between audio tracks. The original song and its karaoke version are expected to show strong correlation because they share the same underlying musical structure (melody, harmony, rhythm), differing mainly in the presence or absence of vocals. In contrast, a completely different song shares none of this structure, so its correlation with either the original or the karaoke track is expected to remain low across all measures.

More broadly, this demonstrates why raw waveform correlation alone is not always sufficient for audio similarity tasks — it is sensitive to phase, timing, and loudness — while perceptual features like Chroma and MFCC better reflect what a human listener would judge as 'the same song' versus 'a different song.' This same approach underlies practical applications such as cover-song detection, audio fingerprinting/identification, and synchronizing vocal and instrumental tracks.

Note: Replace the placeholder screenshot boxes above with your own output once you run the script on your actual audio files — the exact correlation values will depend on the specific tracks used, but the pattern (high similarity for Original-Karaoke, low similarity for the unrelated song) should hold.